Virtualization and Cloud Computing
Assignment 2: VM Auto Scaling and Security on GCP

Name: Om Kumar (B22CS081)
Instructor: Prof. Sumit Kalra
Cloud Platform Used: Google Cloud Platform (GCP)

1 Introduction

In this assignment, I worked with a public cloud platform to practically understand how virtual machines can be deployed, automatically scaled based on workload, and secured using access control and network security mechanisms. One of the key advantages of cloud computing is **auto-scaling**, where compute resources dynamically increase or decrease depending on real-time demand.

In this work, I implemented a complete virtual machine setup on Google Cloud Platform (GCP). I created a project, configured a virtual machine template, enabled CPU-based auto-scaling using a managed instance group, and applied security controls using Identity and Access Management (IAM) and firewall rules. Finally, I tested the setup by generating CPU load and observing the automatic scaling behavior.

2 Objective

The objectives of this assignment are:

- To create a virtual machine environment using a public cloud platform.
- To configure auto-scaling policies that dynamically increase or decrease VM instances based on CPU utilization.
- To apply security mechanisms such as:
  - IAM roles for restricted access control.
  - Firewall rules to allow or deny incoming network traffic.
- To test and validate the auto-scaling behavior through practical experimentation.

3 Cloud Platform Selection

I chose Google Cloud Platform (GCP) for this assignment because student credits were provided through the course. Additionally, GCP provides built-in support for Managed Instance Groups, auto-scaling policies, monitoring services, IAM, and firewall rules, which makes it well suited for demonstrating all the requirements of this assignment without using additional third-party tools.

4 Implementation Details

4.1 Project Creation and Initial Setup

I began by logging into the Google Cloud Console and creating a new project named **vcc-autoscaling-om**. Creating a dedicated project ensured that all resources related to this assignment were isolated from other cloud workloads and could be managed and monitored independently. For deployment, I selected the **asia-south1** region, as it is geographically close to

Jodhpur, resulting in lower network latency when accessing the virtual machines. While other regions may offer slightly lower estimated monthly costs, latency considerations were prioritized for this demonstration. The managed instance group was deployed in the **asia-south1-c** zone within this region.

After the project creation, I navigated to *APIs & Services* and enabled the following APIs:

- **Compute Engine API**: This API is required to create, configure, and manage virtual machine instances, instance templates, and managed instance groups.
- **Cloud Monitoring API**: This API is essential because auto-scaling decisions are based on monitored CPU utilization metrics collected over time.

Enabling these APIs is a prerequisite for implementing CPU-based auto-scaling in GCP.

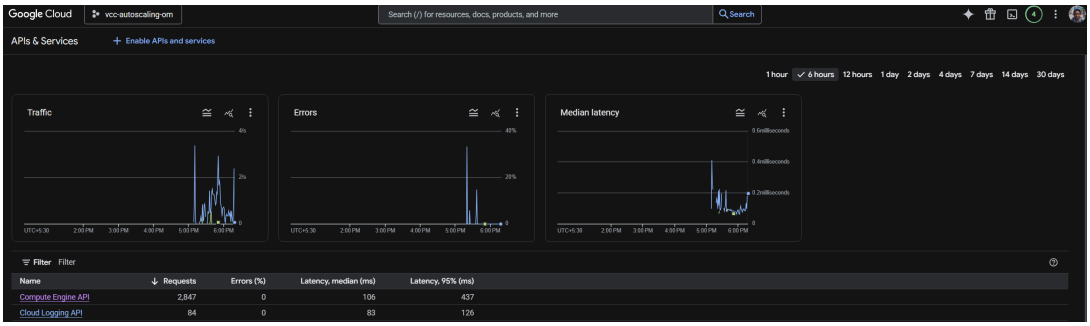


Figure 1: Enabled APIs for Compute Engine and Monitoring

4.2 Virtual Machine Configuration Using Instance Template

Instead of deploying a single standalone virtual machine, I created an **Instance Template** named **om-vcc-template**. An instance template defines a standardized and reusable configuration that is used to create identical VM instances automatically during scaling events. This approach is mandatory because GCP auto-scaling operates only on managed instance groups, not on individual VMs.

The instance template was created with the following configuration:

- **Location**: asia-south1. A regional template was used so that instances could be created reliably within the selected region.
- **Machine Type**: e2-medium. I selected the E2 machine family because it provides cost-optimized, general-purpose performance suitable for auto-scaling demonstrations. While alternatives such as N2/N1 (general-purpose), C2 (compute-optimized), and M1/M2 (memory-optimized) machines are available, they were unnecessary for this workload and would increase cost without additional benefit.
- **Operating System**: Ubuntu 22.04 LTS Minimal (x86/64), chosen for its long-term support, stability, and minimal resource footprint.
- **Boot Disk**: 10 GB Balanced Persistent Disk, which provides sufficient performance for a lightweight web server while remaining cost-efficient.
- **Networking**: Default VPC network with an ephemeral external IP address.
- **Firewall**: HTTP traffic enabled and HTTPS disabled, as the assignment required only basic web access.

I deliberately kept advanced features such as GPUs, confidential computing, and custom placement policies disabled, as they are unnecessary for demonstrating auto-scaling and security concepts in this assignment.

To ensure automatic configuration of every VM instance created from the template, I added the following startup script:

```
#!/bin/bash
apt update
apt install -y apache2 stress
echo "<h1>VCC_Assignment_2_Auto_Scaling_VM</h1>" > /var/www/html/index.
    html
systemctl start apache2
```

This startup script installs the Apache web server to serve HTTP requests and the `stress` utility, which is later used to generate CPU load during testing.

Figure 2: Instance Template `om-vcc-template` Configuration

4.3 Managed Instance Group Creation

After creating the instance template, I created a **stateless Managed Instance Group (MIG)** named **om-vcc-mig** using the previously defined template. The managed instance group was deployed in the **asia-south1-c** zone.

A stateless configuration was selected because auto-scaling works most effectively when VM instances do not maintain persistent local state. This allows instances to be created or deleted freely during scale-up and scale-down events without affecting application correctness.

The managed instance group was configured as follows:

- **Initial number of instances**: 1, ensuring minimum resource availability.
- **Instance source**: Existing instance template (om-vcc-template).
- **Autohealing**: Enabled with default repair action, allowing GCP to automatically recreate unhealthy instances.
- **Predictive autoscaling**: Disabled, so that scaling decisions depend only on real-time CPU metrics.

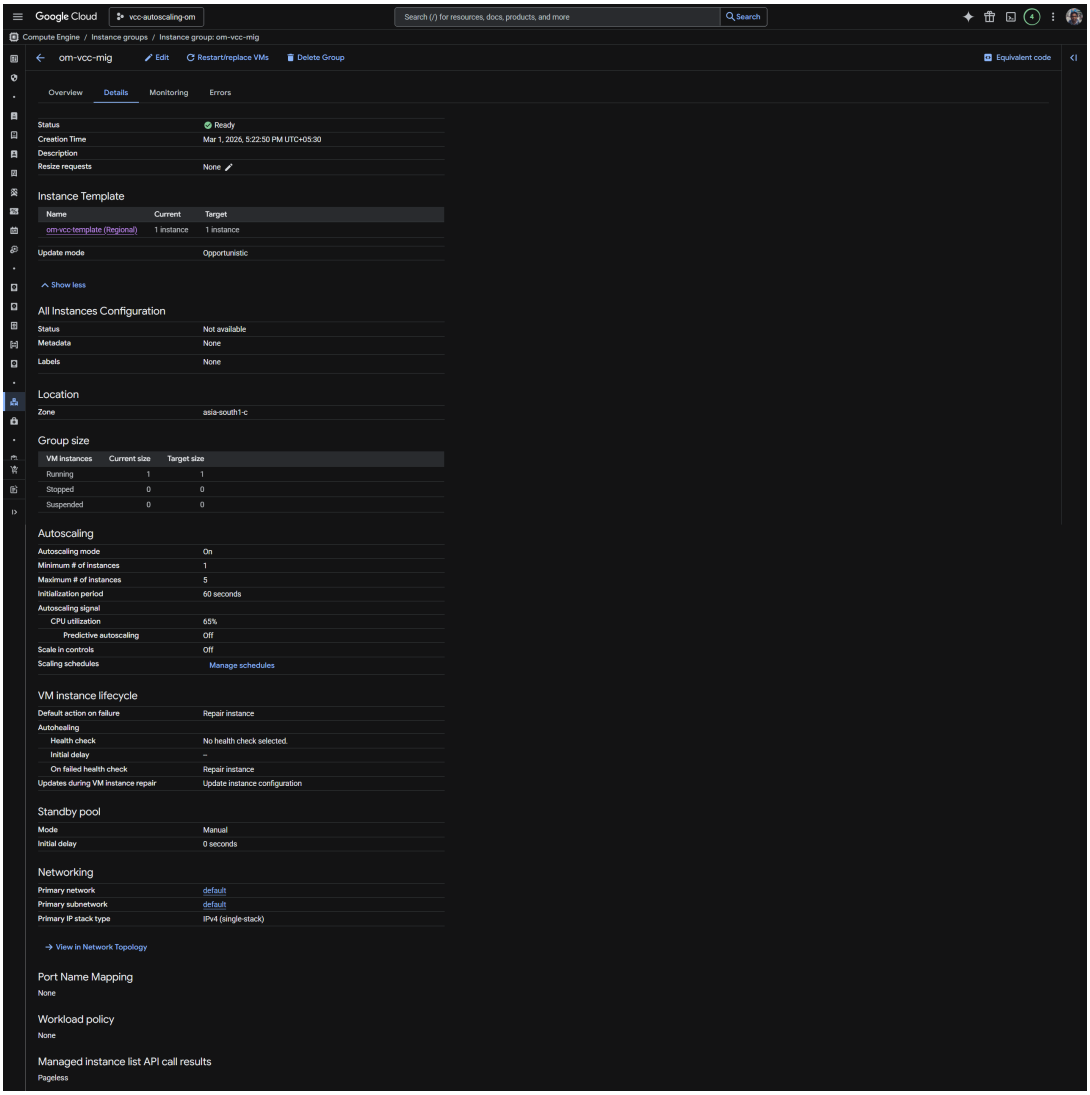


Figure 3: Managed Instance Group `om-vcc-mig` in asia-south1-c

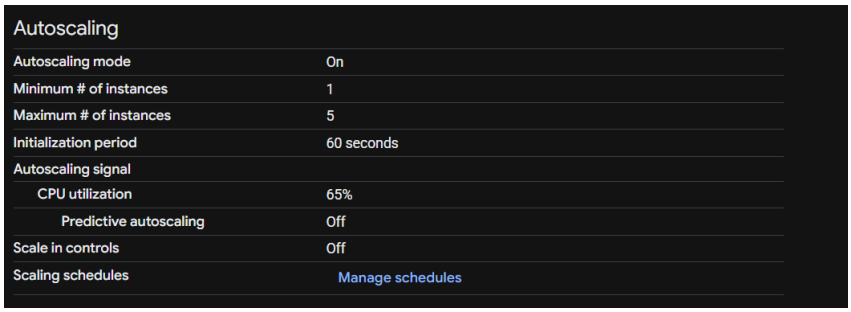
4.4 Auto-Scaling Policy Configuration

Once the managed instance group was created, I enabled auto-scaling based on CPU utilization. CPU utilization was selected as the scaling signal because the workload is CPU-bound, and increased demand directly results in higher processor usage. While HTTP load balancing utilization is another possible signal, it requires a load balancer and is more suitable for request-based scaling, which was not required for this assignment.

The auto-scaling configuration was set as follows:

- **Autoscaling metric**: CPU utilization
- **Target CPU utilization**: 65%. This threshold provides a balance between responsiveness and cost efficiency. Lower values may lead to premature scaling and increased cost, while higher values could cause performance degradation before new instances are added.
- **Minimum instances**: 1
- **Maximum instances**: 5
- **Initialization period**: 60 seconds

Predictive autoscaling and scale-in controls were kept disabled to maintain a simple and transparent scaling behavior suitable for demonstration purposes.



Autoscaling	
Autoscaling mode	On
Minimum # of instances	1
Maximum # of instances	5
Initialization period	60 seconds
Autoscaling signal	
CPU utilization	65%
Predictive autoscaling	Off
Scale in controls	Off
Scaling schedules	Manage schedules

Figure 4: CPU-Based Auto-Scaling Policy Configuration

5 Security Implementation

5.1 Identity and Access Management (IAM)

To implement restricted access, I configured IAM roles following the principle of least privilege. I added a secondary user (omkumar70152@gmail.com) and assigned the **Compute Viewer** role.

This role allows the user to view virtual machine and instance group details but does not allow creating, modifying, or deleting any resources, thereby ensuring controlled access.

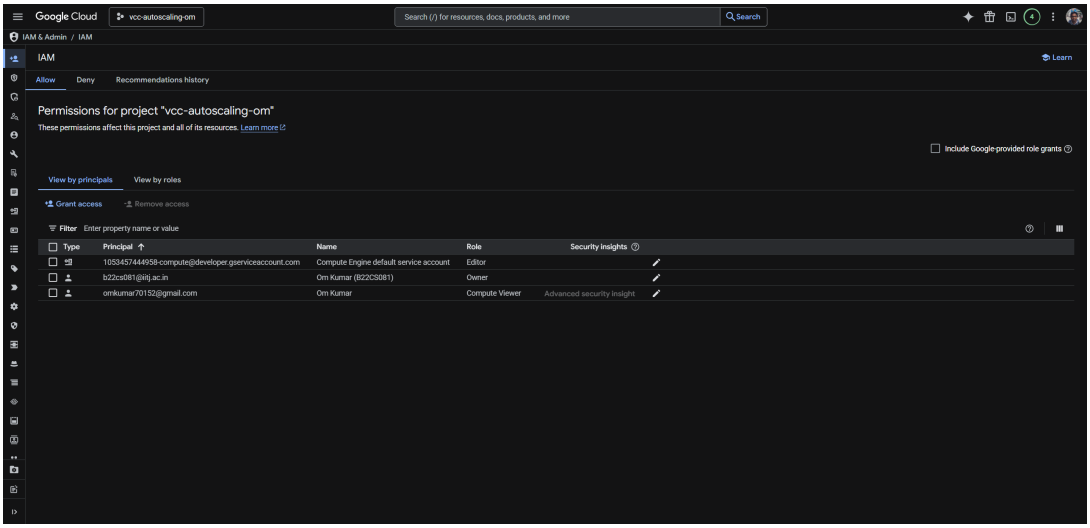


Figure 5: IAM Role Assignment

5.2 Firewall Rules Configuration

I configured firewall rules to control inbound network traffic to the VM instances. The direction of traffic was set to **Ingress**, as the goal was to regulate incoming connections.

An ingress firewall rule named **allow-http-om** was created to allow HTTP traffic on TCP port 80 from all sources (0.0.0.0/0), enabling public access to the web server running on the virtual machines. SSH access on TCP port 22 was restricted to my public IP address only to prevent unauthorized access.

The firewall rules were applied to **all instances in the default VPC network**, ensuring that any new VM instances created automatically during auto-scaling inherit the same security policies.

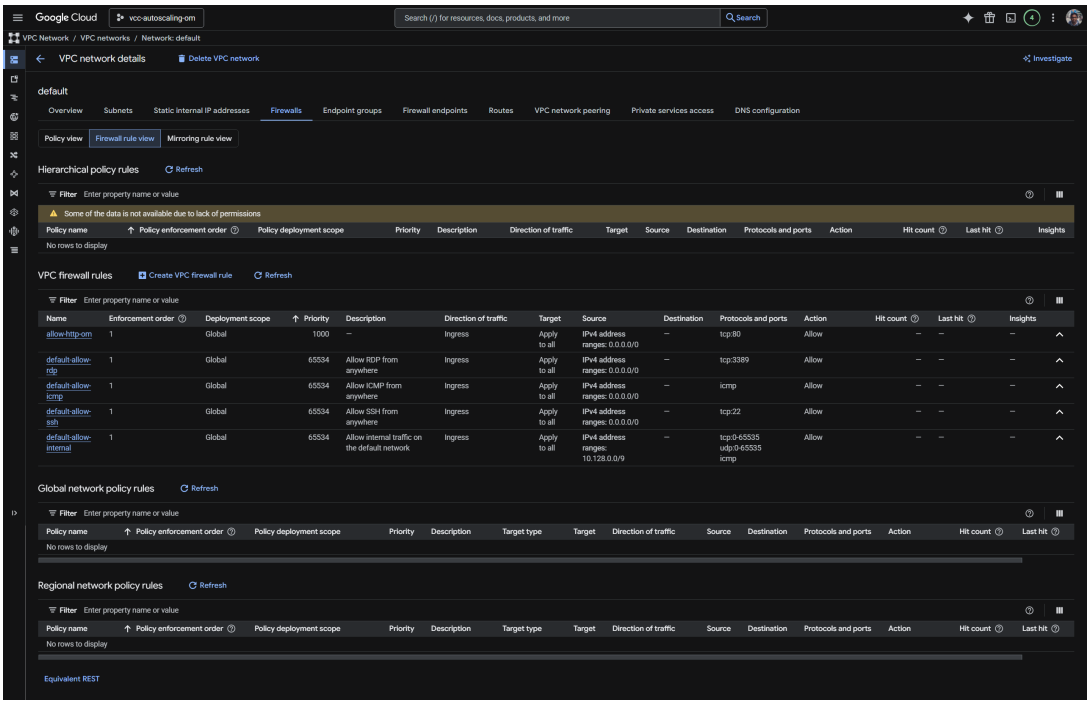


Figure 6: Firewall Rules Configuration

6 Testing and Validation

To validate the auto-scaling behavior, I connected to one of the VM instances using SSH and executed the following command to generate CPU load:

```
stress --cpu 4
```

As the CPU utilization exceeded the 65% threshold, the managed instance group automatically created additional VM instances. The scale-up process occurred almost immediately in order to handle the increased load.

After stopping the stress command, CPU utilization dropped below the threshold. However, the additional instances were not terminated immediately. The scale-down process occurred approximately 7 to 8 minutes later due to the initialization period and stabilization logic. This behavior is intentional and helps prevent rapid oscillations between scaling up and scaling down.

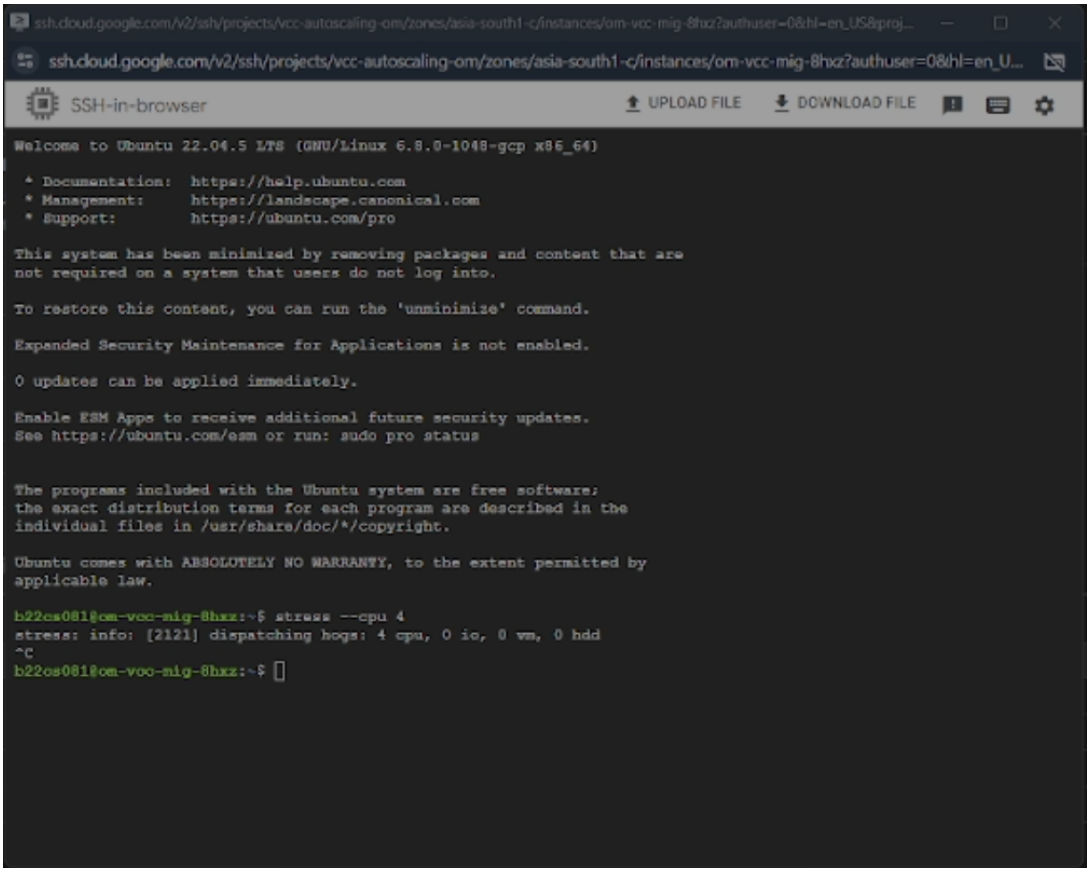


Figure 7: CPU Load Generation and Instance Scale-Up

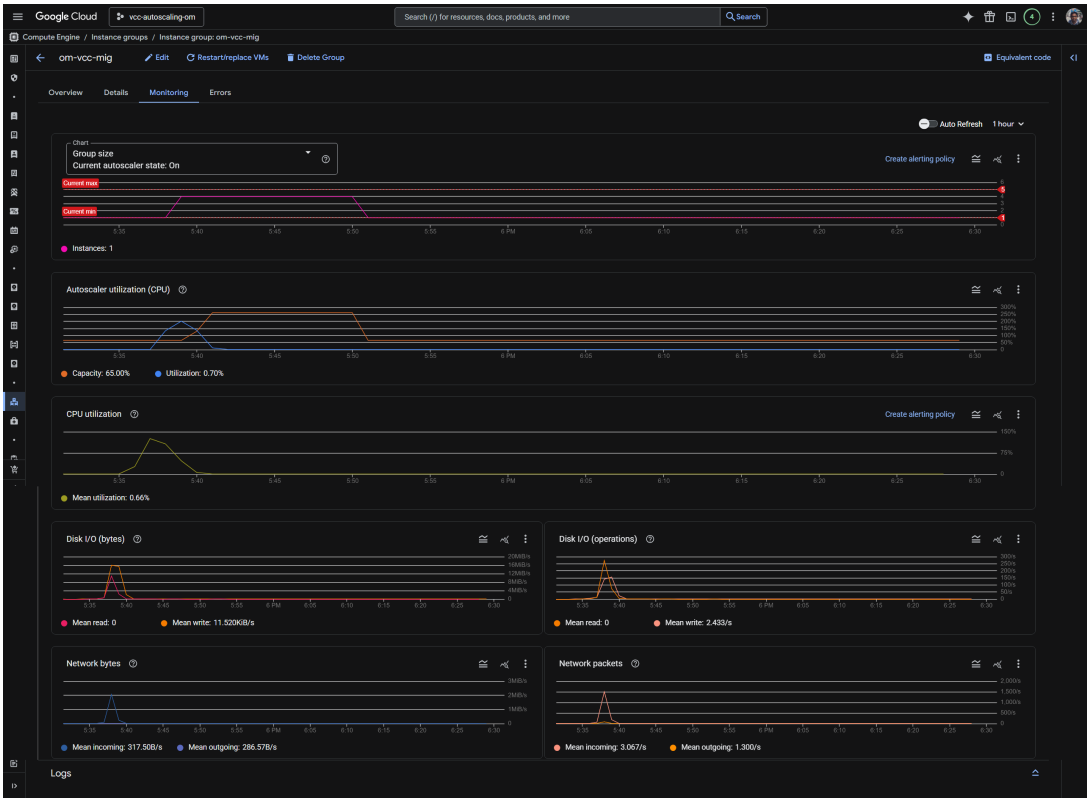


Figure 8: CPU Utilization Monitoring During Scale-Down

7 Architecture Design

The architecture of the implemented system is designed to demonstrate automatic scaling of virtual machines along with essential security controls using Google Cloud Platform services. The overall architecture is centered around Google Cloud Compute Engine and its native auto-scaling capabilities.

At the core of the architecture is **Google Cloud Compute Engine**, which provides the infrastructure for creating and managing virtual machine instances. Instead of deploying individual standalone VMs, an **Instance Template** (`om-vcc-template`) is used to define a standardized configuration for all virtual machines. This template specifies the machine type (e2-medium), operating system (Ubuntu 22.04 LTS Minimal), boot disk, networking configuration, and startup script. Using an instance template ensures that every VM created during scaling events is identical and consistently configured.

A **stateless Managed Instance Group (MIG)** named `om-vcc-mig` is created from the instance template and deployed in the **asia-south1-c** zone. The managed instance group is responsible for automatically creating, deleting, and maintaining the required number of VM instances. Stateless configuration was chosen to allow instances to be freely added or removed without maintaining persistent local state, which is ideal for auto-scaling environments.

An **auto-scaling policy** is attached to the managed instance group. This policy uses **CPU utilization** as the scaling signal. When the average CPU utilization across instances exceeds the target threshold of **65%**, the auto-scaler automatically increases the number of VM instances, up to a maximum of five. When CPU utilization drops below the threshold for a sustained period, the auto-scaler gradually reduces the number of instances down to the minimum of one. An initialization period of 60 seconds is configured to prevent frequent scaling oscillations.

Security is enforced at multiple layers in the architecture. **Identity and Access Management (IAM)** policies restrict access to Compute Engine resources. A secondary user is

granted the Compute Viewer role, allowing read-only access while preventing any modifications to the infrastructure.

At the network level, **firewall rules** are applied within the default VPC network to control inbound traffic. Ingress rules allow HTTP traffic on port 80 from all sources to enable public access to the web server, while SSH access on port 22 is restricted to the administrator's public IP address. These firewall rules apply to all instances in the network, ensuring that any new VM created during scaling events automatically inherits the same security policies.

Network traffic flow follows a clear path: inbound client requests first pass through the firewall rules defined in the VPC network. Only permitted traffic is allowed to reach the virtual machine instances within the managed instance group. CPU usage generated by the workload is continuously monitored by Cloud Monitoring services, which feed metrics to the auto-scaling mechanism to dynamically adjust the number of running instances.

This architecture provides a scalable, secure, and cost-efficient virtual machine environment that dynamically adapts to workload demands while maintaining controlled access and network security.

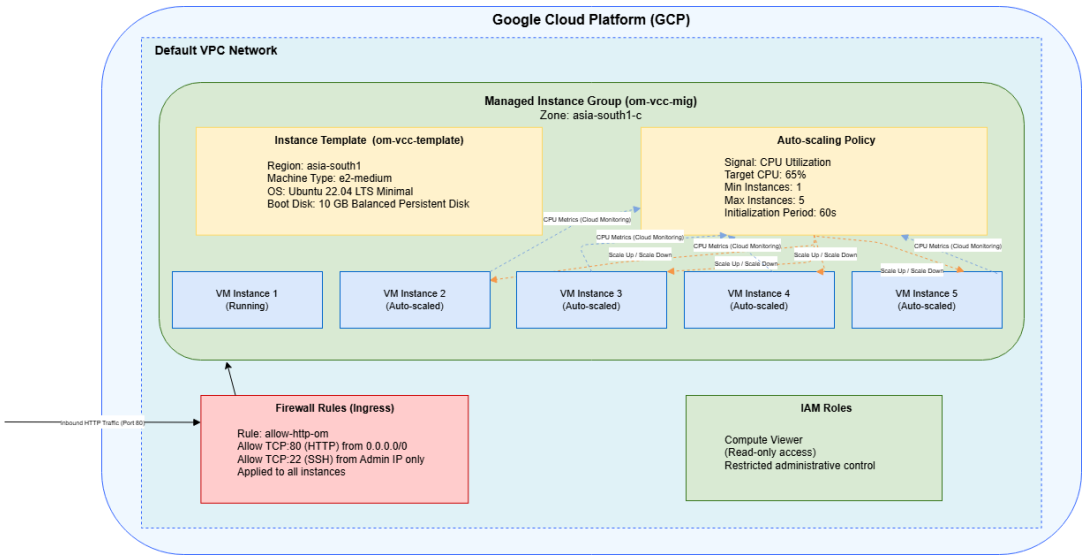


Figure 9: Overall Architecture of Auto-Scaling and Security Setup

8 Source Code Repository

The source code repository contains the startup script and supporting documentation used in this assignment.

`https://github.com/OmKumar2004/vcc-assignment-2-autoscaling`

9 Recorded Video Demonstration

I recorded a video demonstrating the complete setup process, auto-scaling behavior, and security configuration.

`https://drive.google.com/file/d/1U9MbV9XkhbjAvFbkCLZfXLKltHAyN69u/view?usp=sharing`

10 Conclusion

In this assignment, I successfully implemented a scalable and secure virtual machine environment on Google Cloud Platform. I demonstrated CPU-based auto-scaling using a managed

instance group and validated both scale-up and scale-down behavior. I also applied IAM roles and firewall rules to secure the infrastructure. This assignment provided hands-on experience with real-world cloud scalability and security concepts.

11 Plagiarism Declaration

I hereby declare that this assignment is entirely my own work. I have not copied this work from other students. I referred to official documentation and online resources only for understanding the steps involved. Any instance of plagiarism may result in the submission being declared void, as per university regulations.